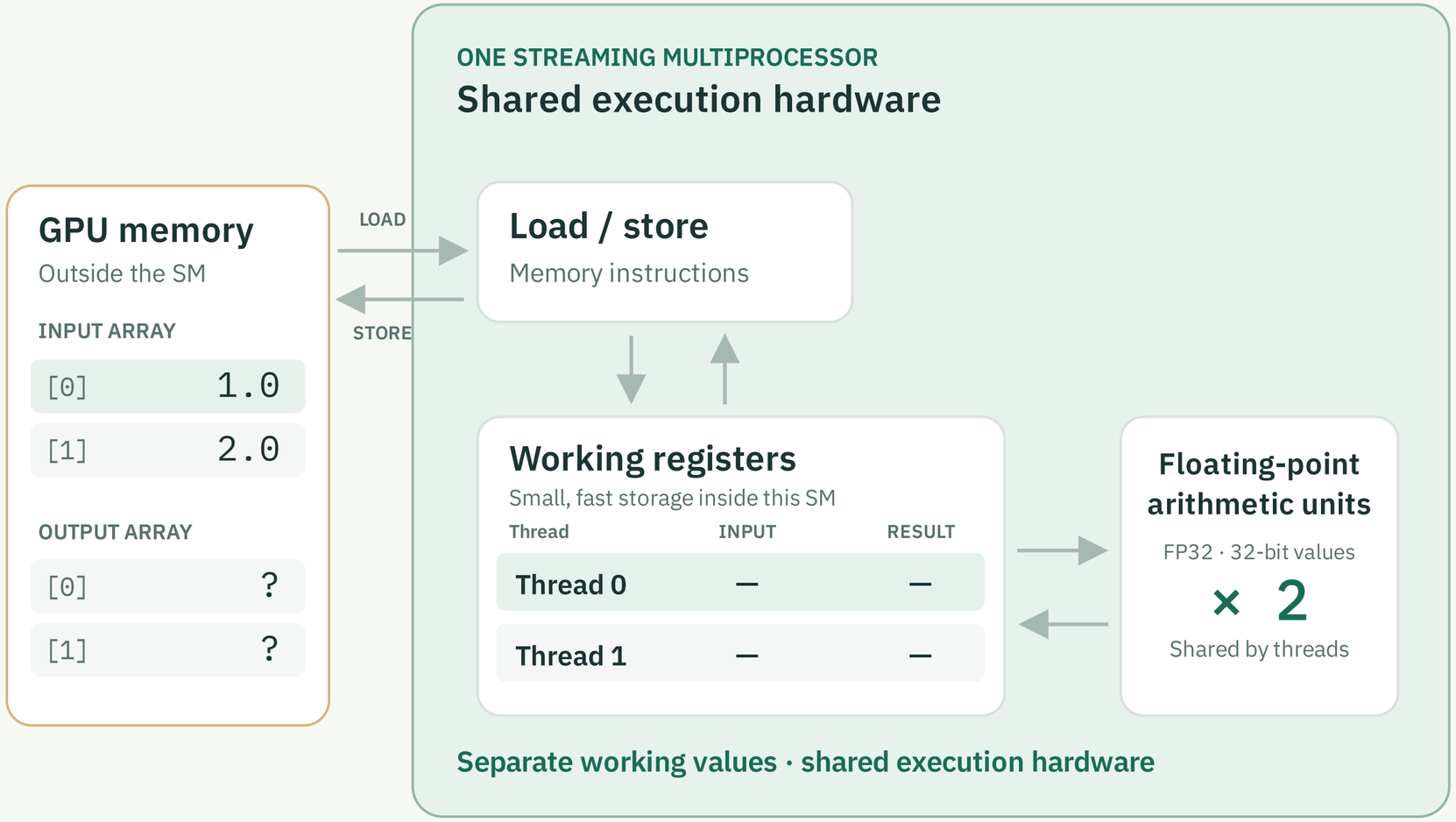
THE QUESTION

Where does the multiplication happen?

Follow 1.0 → 2.0 inside the GPU.

PyTorch operation

```
y = x * 2
```



Conceptual value path. Arrows do not show elapsed time; caches and unit counts are omitted.

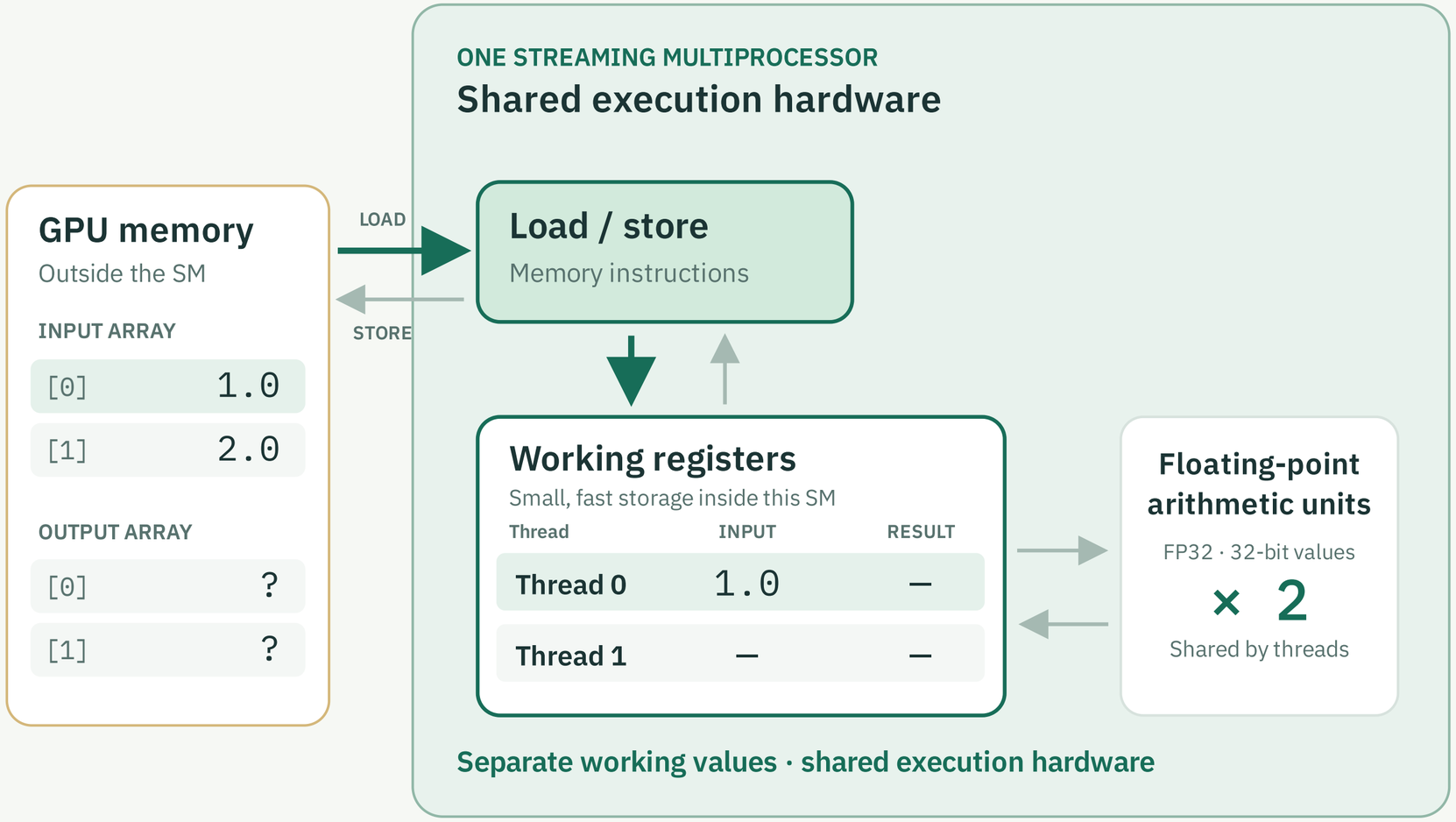
01 · LOAD

Load the input.

A register is small, fast storage inside the SM.

Request → arrival

Multiply waits for its input



Conceptual value path. Arrows do not show elapsed time; caches and unit counts are omitted.

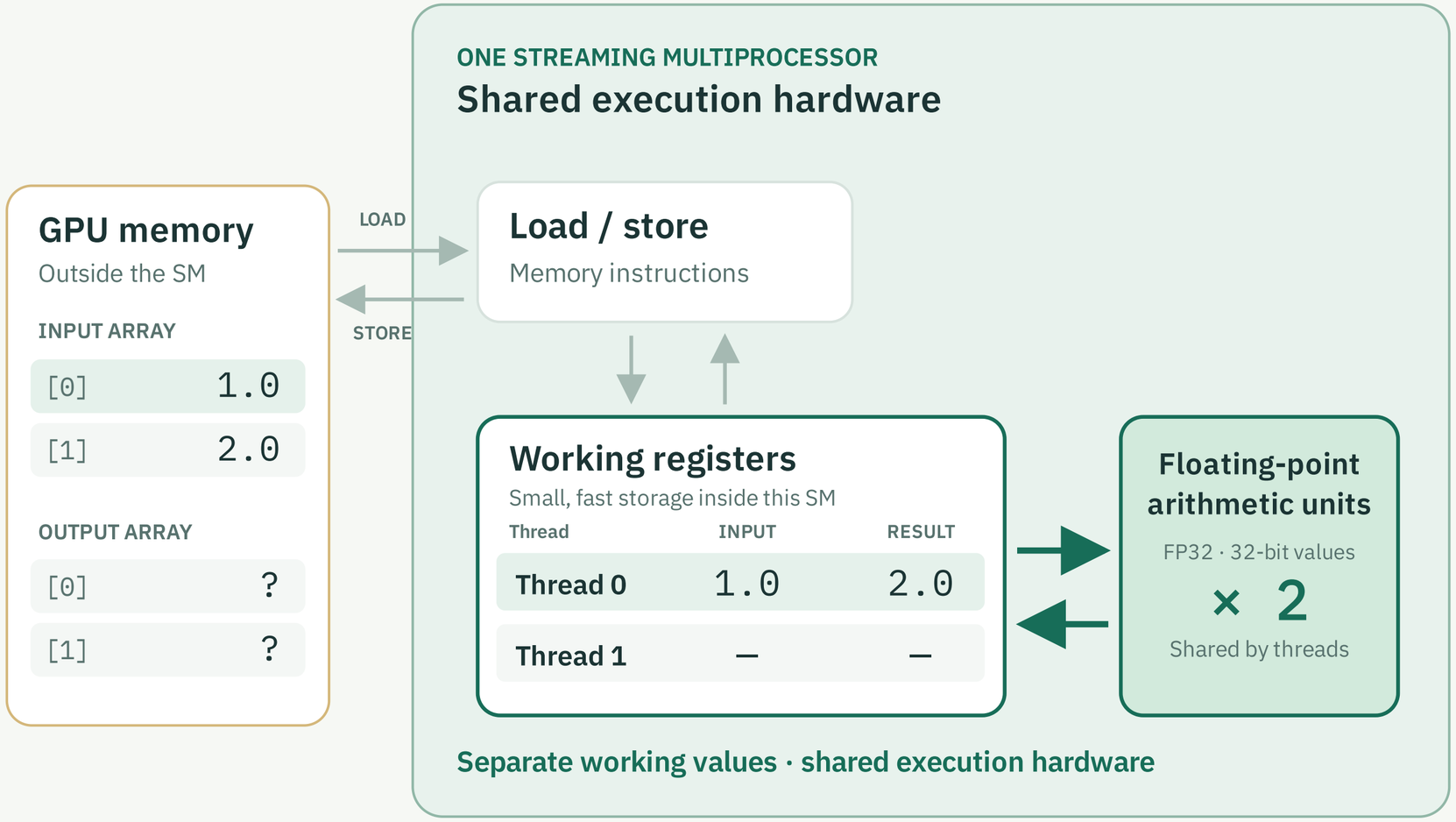
02 · MULTIPLY

Multiply the register value.

The result returns to a register.
Output memory is still unwritten.

1.0 × 2 → 2.0

The store has not run



Conceptual value path. Arrows do not show elapsed time; caches and unit counts are omitted.

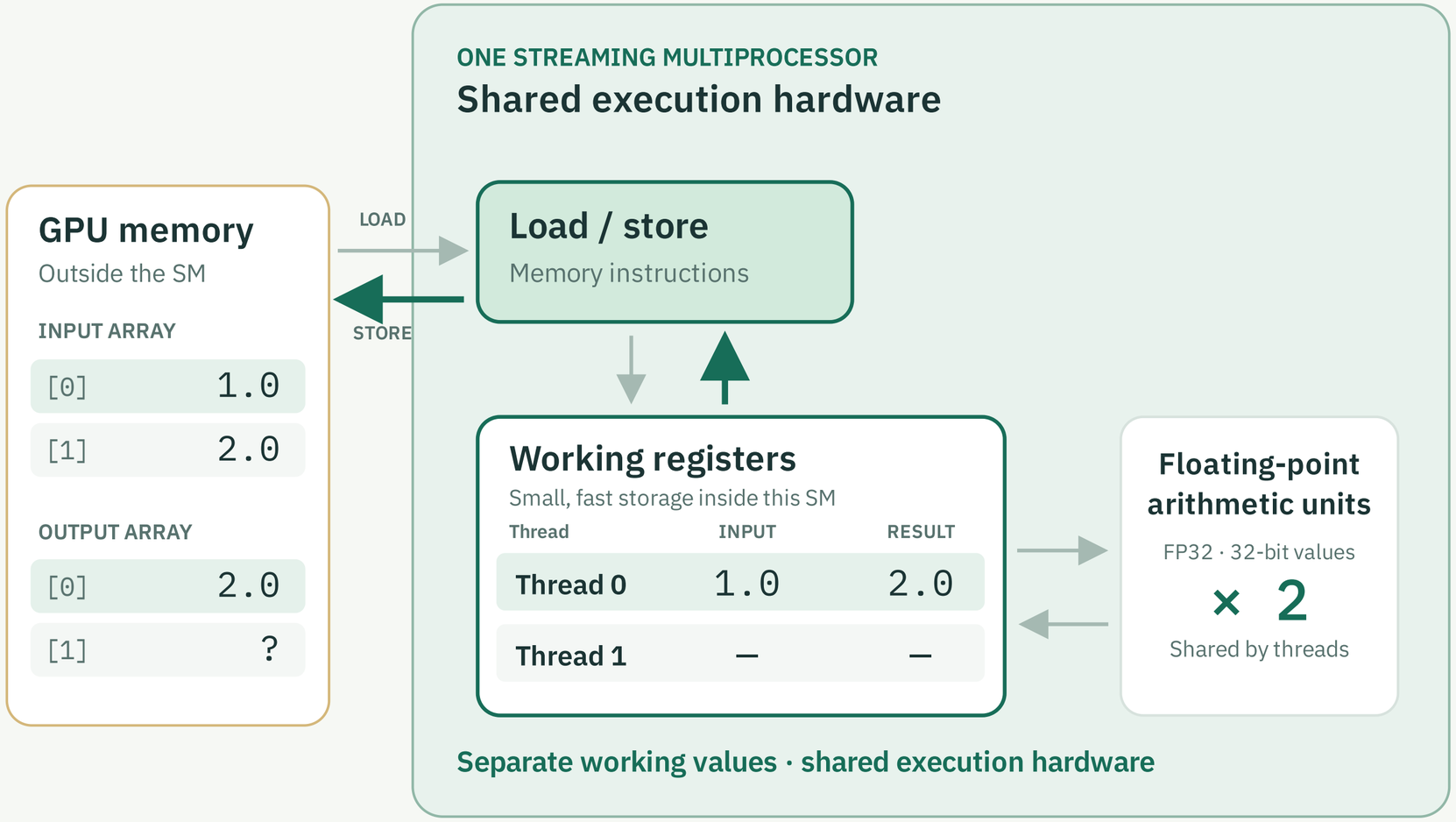
03 · STORE

Write the result back.

The store writes 2.0 into output[0].

Register → memory

output[0] now holds 2.0



Conceptual value path. Arrows do not show elapsed time; caches and unit counts are omitted.

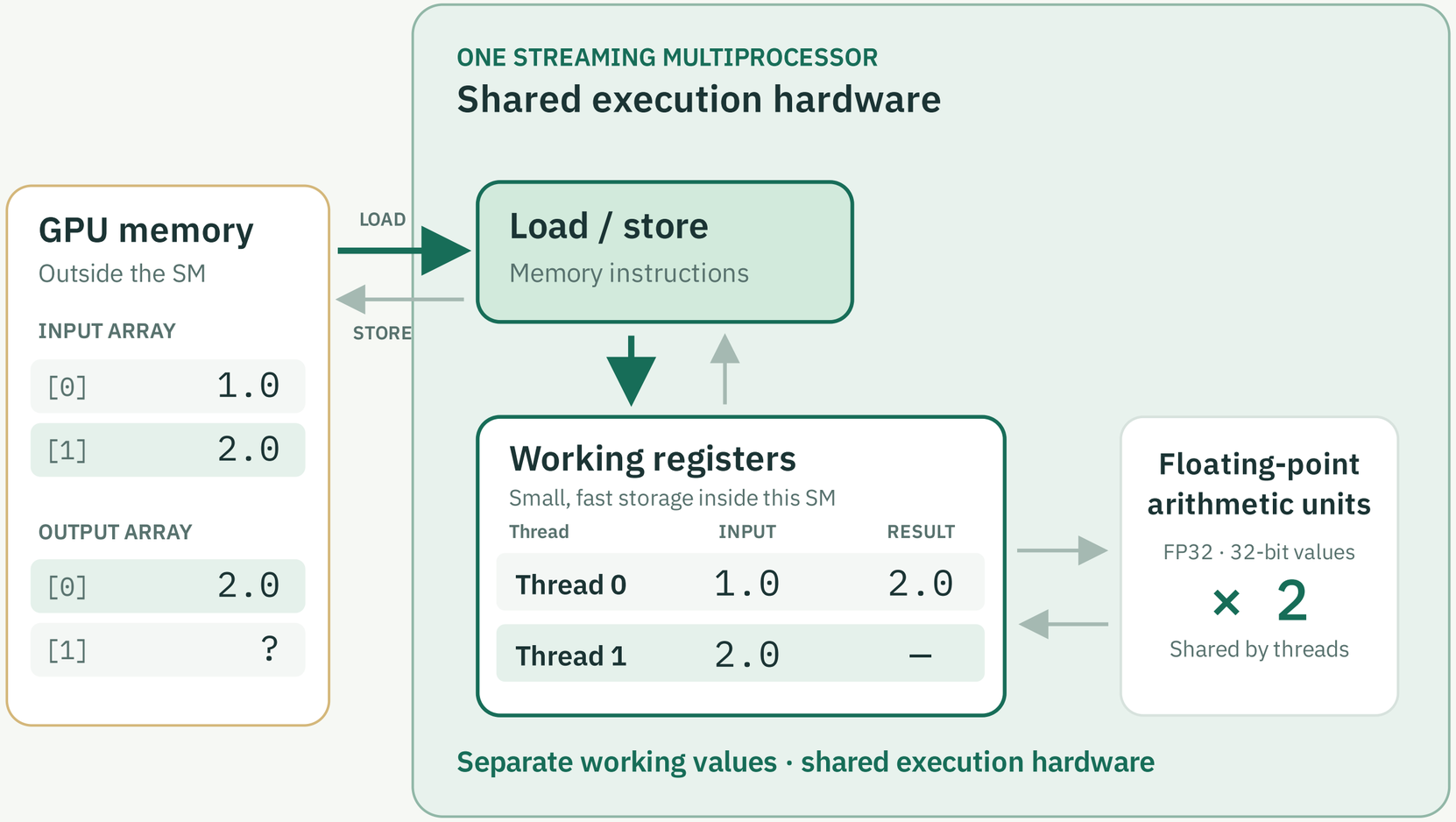
ANOTHER THREAD

Another thread. Separate values.

Thread 1 loads 2.0 while Thread 0's values remain distinct.

Different working values

The same kinds of hardware



Conceptual value path. Arrows do not show elapsed time; caches and unit counts are omitted.

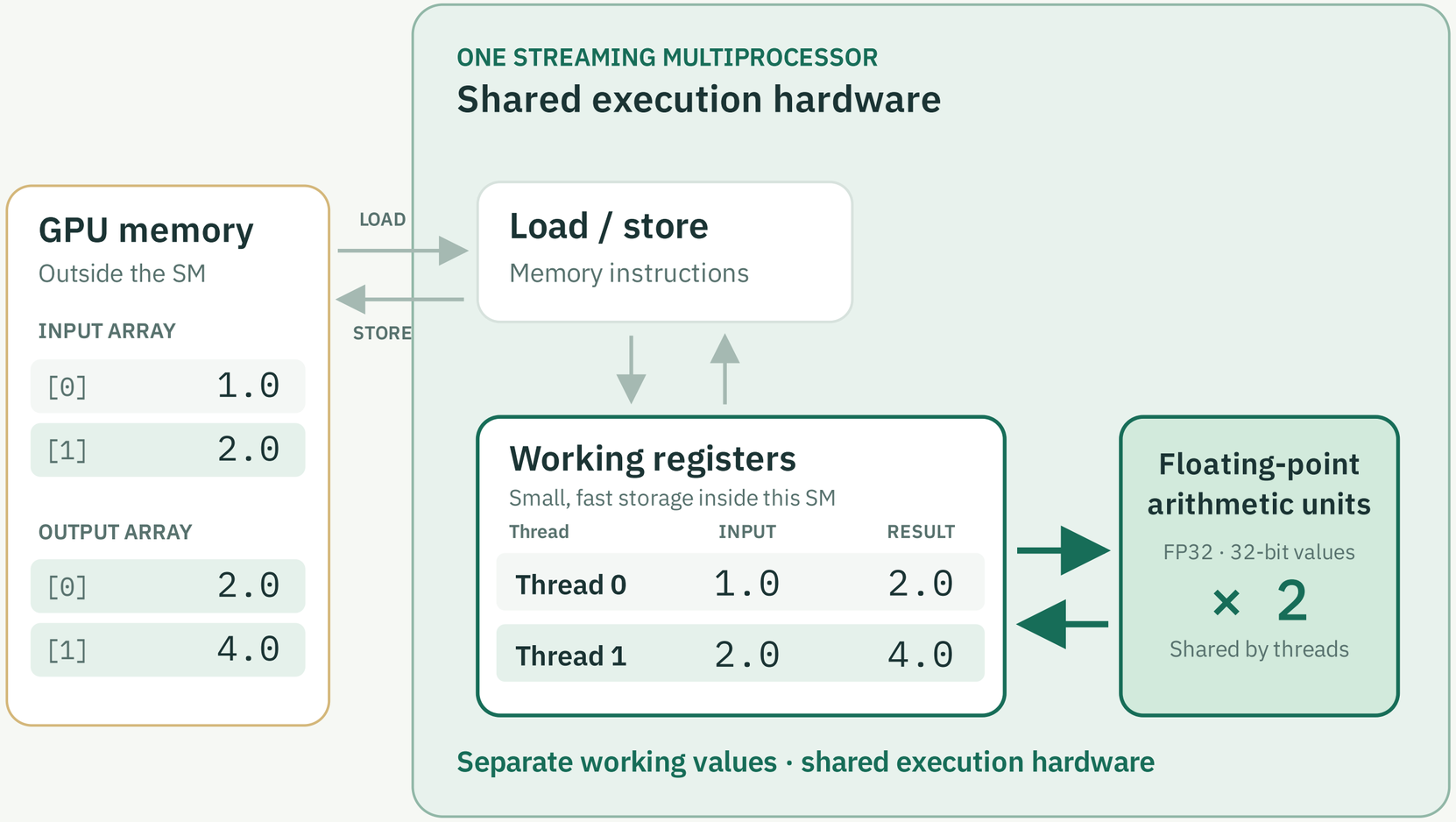
THE DISTINCTION

The hardware is shared.

Thread 1 multiplies 2.0 → 4.0, then stores 4.0.

Separate values. Shared units.

Registers can be reused



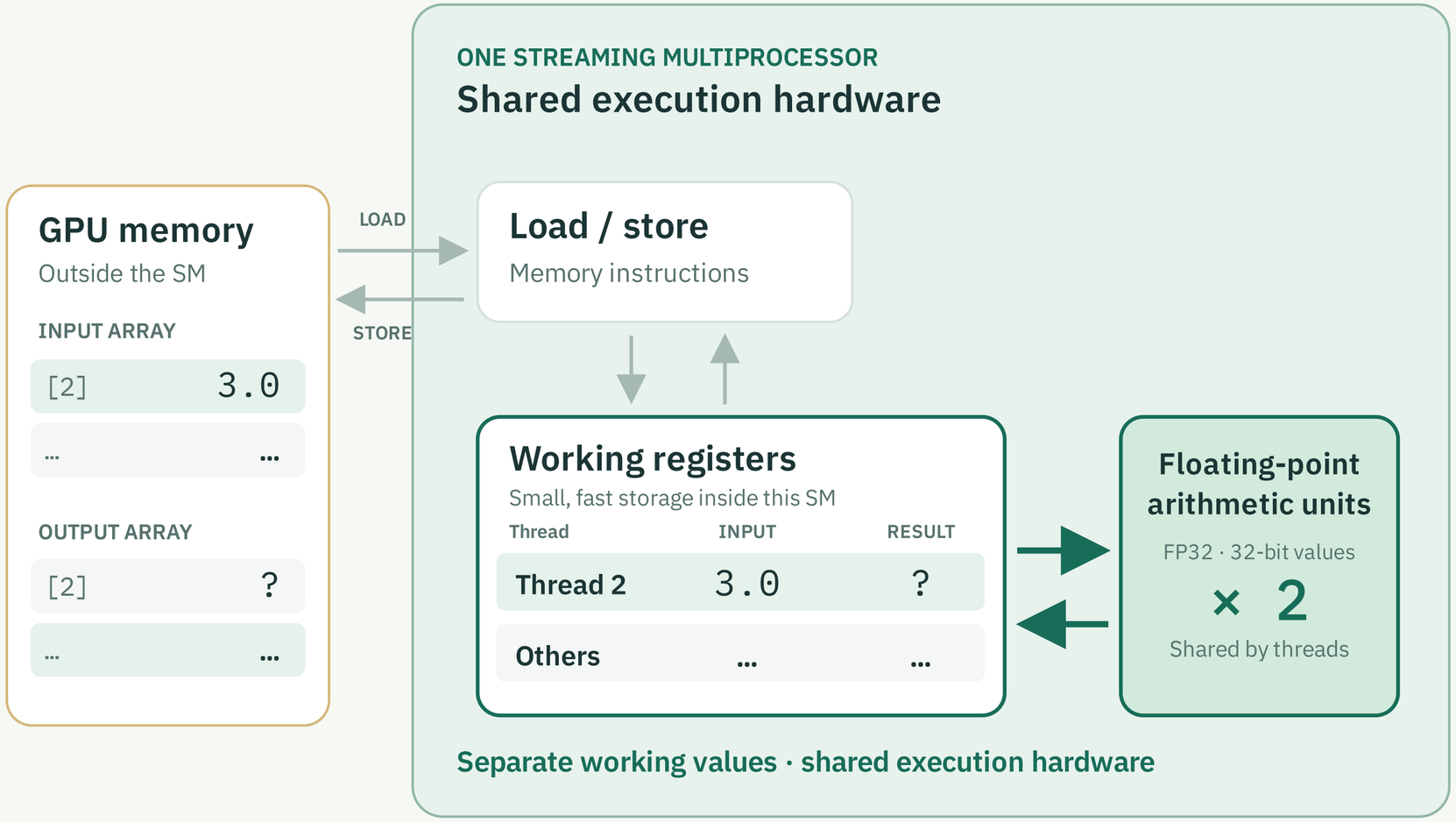
Conceptual value path. Arrows do not show elapsed time; caches and unit counts are omitted.

YOUR TURN

Pause and explain.

A third thread loads 3.0 and multiplies by 2. The store has not run.

> Where is its result: a working register or the output array?
>
> Pause the narration to think.



Conceptual value path. Arrows do not show elapsed time; caches and unit counts are omitted.

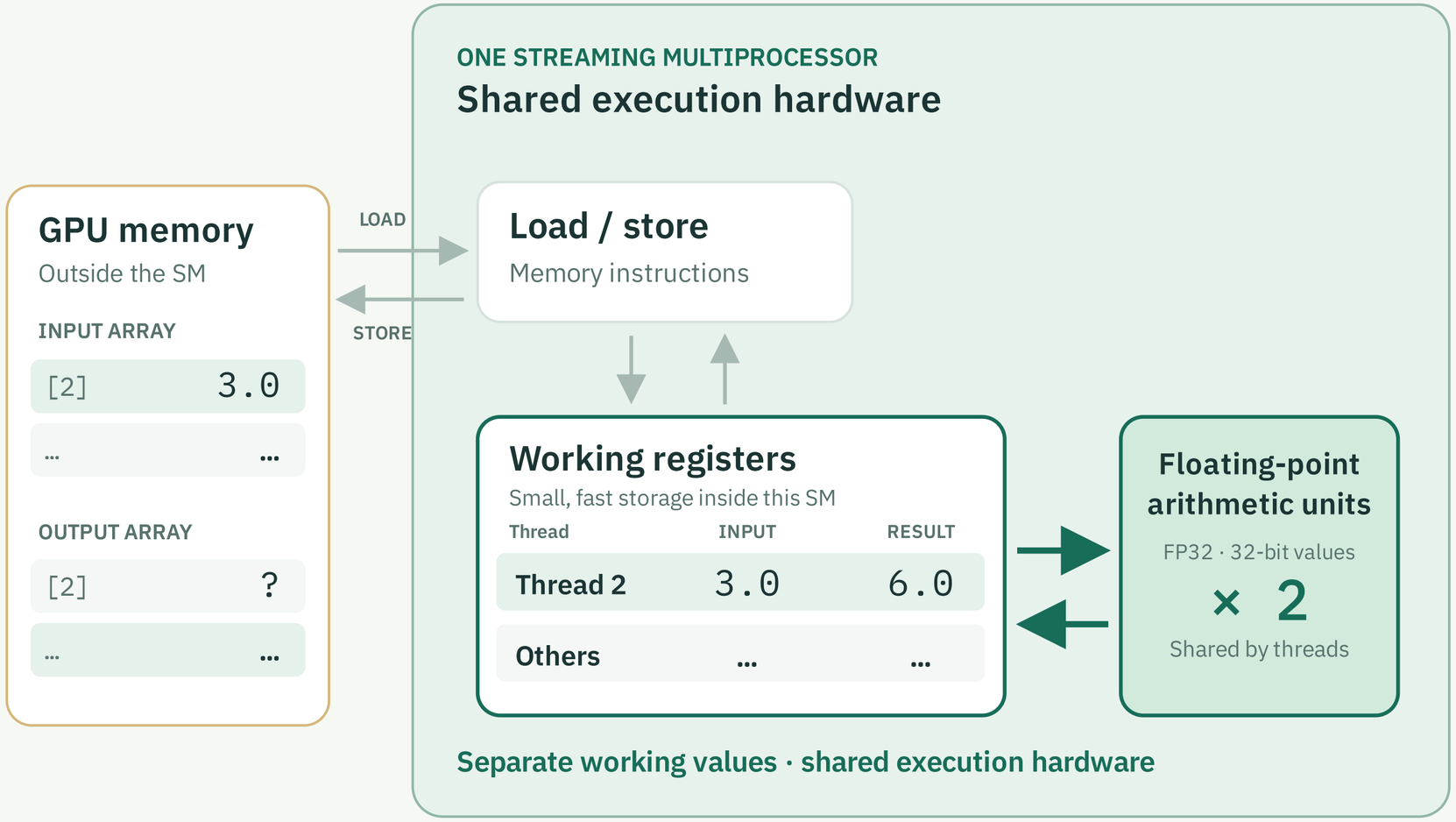
TAKE IT BACK

Registers first. Memory after the store.

The multiply produces 6.0 in a working register.

Load → multiply → store

A worked illustration, not PyTorch's instruction trace



Conceptual value path. Arrows do not show elapsed time; caches and unit counts are omitted.